CS24303 • MODULE 3

Module 3: Data Warehousing & OLAP Technology

Topics 21 to 29 • Star/Snowflake Schemas, Measures, OLAP Operations, Cuboid Lattices & BUC

Module 3 Table of Contents (Topics 21 to 29)

Topic 21 & 22: Data Warehouse Architecture & OLTP vs. OLAP

According to William H. Inmon (the father of data warehousing), a **Data Warehouse** is defined as: *"A subject-oriented, integrated, time-variant, and non-volatile collection of data in support of management's decision-making process."*

Characteristic	Inmon's Definition	Technical Implementation
Subject-Oriented	Organized around major business subjects (Customer, Product, Sales, Supplier) rather than operational transactional processes (order entry, invoicing).	De-normalized multi-dimensional schema tables optimizing cross-departmental queries.
Integrated	Constructed by integrating heterogeneous sources (relational DBs, flat files, legacy systems). Naming inconsistencies, encoding formats, and conflicting measurement units are unified.	ETL (Extract, Transform, Load) staging pipelines enforcing global enterprise metadata standards.
Time-Variant	Every key record in the data warehouse explicitly contains a time dimension key (Day, Month, Quarter, Year), capturing historical snapshots across a 5–10 year horizon.	Slowly Changing Dimensions (SCD Type 1, 2, 3), append-only historical fact tables.
Non-Volatile	Data is never updated or deleted in place by end-users. Once loaded into the warehouse, records remain static and read-only.	Periodic batch loads; high-speed read-optimized indexing (Bitmap, B-tree indexes).

Topic 23 & 24: Multidimensional Schemas & Categorization of Measures

Figure 3.1: The Star Schema Architecture (Central Fact Table radiating to Dimension Tables)

Schema Architecture	Structural Design	Key Advantages & Trade-offs
Star Schema	A single central fact table referencing multiple completely de-normalized dimension tables via foreign keys.	Extremely simple queries; minimal SQL joins; redundant storage in dimension tables.
Snowflake Schema	A refinement of the star schema where dimension tables are normalized into hierarchies (splitting `DIM_ITEM` into `ITEM`, `CATEGORY`, `SUPPLIER`).	Zero data redundancy; saves disk storage; complex multi-table SQL joins reduce OLAP query performance!
Fact Constellation (Galaxy)	Multiple distinct fact tables (e.g. `FACT_SALES` and `FACT_SHIPPING`) sharing common dimension tables (Conformed Dimensions).	Enterprise-wide standard supporting complex business intelligence workflows across multiple functional divisions.

Categorization of Multidimensional Measures: - **Additive Measures:** Can be meaningfully aggregated across ALL dimensions (e.g., `units_sold`, `dollars_revenue`, `cost_amount`). - **Semi-Additive Measures:** Can be aggregated across some dimensions, but NOT across the Time dimension (e.g., `bank_account_balance`, `warehouse_inventory_count` — summing balances across 365 days gives a meaningless number!). - **Non-Additive Measures:** Cannot be aggregated across any dimension by simple addition (e.g., `unit_price`, `profit_margin_%`, `temperature_celsius`). Must compute aggregate numerator and aggregate denominator separately ($\text{AvgMargin} = \frac{\sum \text{Profit}}{\sum \text{Revenue}}$).

Topic 25 to 27: OLAP Operations & Data Cube Computation

OLAP Operation	Algorithmic Transformation	Business Analytics Example
Roll-Up (Drill-Up)	Performs aggregation on a data cube by climbing up a concept hierarchy or by dimension reduction (dropping a dimension).	Aggregating daily sales records into quarterly totals: `Day` $\rightarrow$ `Month` $\rightarrow$ `Quarter` $\rightarrow$ `Year`.
Drill-Down (Roll-Down)	Navigates from less detailed (high-level summary) data to highly detailed granular data by descending a concept hierarchy or introducing an additional dimension.	De-aggregating national quarterly revenue down to city-level store performance: `Country` $\rightarrow$ `State` $\rightarrow$ `City` $\rightarrow$ `Store`.
Slice	Performs a selection on one dimension of the given cube, resulting in a sub-cube of lower dimensionality (cutting a 2D slice from a 3D cube).	Filtering the sales cube strictly for `Time = "Q1_2024"`.
Dice	Defines a sub-cube by performing a selection on two or more dimensions simultaneously.	Filtering for `(Location = "Toronto" OR "Vancouver") AND (Time = "Q1" OR "Q2") AND (Item = "Electronics")`.
Pivot (Rotate)	Rotates the data axes in space to provide an alternative visual presentation of the multidimensional matrix.	Swapping row axis (`Product`) with column axis (`Quarter`) in a spreadsheet pivot table.

🏛️ Step-by-Step Solved Problem: Cuboid Lattice Combinatorics

An enterprise data warehouse has 4 dimensions: Time (D_1), Item (D_2), Location (D_3), Supplier (D_4).

1. Calculate the total number of cuboids in the full data cube lattice.
2. If dimension Time has hierarchy `Day` $\rightarrow$ `Month` $\rightarrow$ `Quarter` $\rightarrow$ `Year` ($L_1 = 4$ levels + `all`), Item has 3 levels ($L_2 = 3$), Location has 3 levels ($L_3 = 3$), and Supplier has 2 levels ($L_4 = 2$), calculate the total number of cuboids with hierarchies.

Mathematical Solution:

- **1. Without Hierarchies:** A n-dimensional cube contains $\mathbf{2^n}$ cuboids. For $n = 4$:

$$\mathbf{T = 2^4 = 16 \text{ cuboids}}$$

  (From 0-D Apex Cuboid `all` to 4-D Base Cuboid `(Time, Item, Location, Supplier)`).
- **2. With Concept Hierarchies:** Total cuboids $N_{\text{cuboids}} = \prod_{i=1}^{n} (L_i + 1)$:

$$\mathbf{N_{\text{cuboids}} = (4+1) \times (3+1) \times (3+1) \times (2+1) = 5 \times 4 \times 4 \times 3 = 240 \text{ cuboids}}$$

Topic 29: Master University Examination Solved Question Bank

Q1. Compare ROLAP, MOLAP, and HOLAP Architectures. (10 Marks)

Dimension	ROLAP (Relational OLAP)	MOLAP (Multidimensional OLAP)	HOLAP (Hybrid OLAP)
Underlying Storage	Relational DBMS (Star/Snowflake tables).	Proprietary multidimensional arrays (dense matrices).	Relational DB for detailed base data; MOLAP arrays for aggregated summaries.
Scalability	Extremely high (handles petabytes).	Moderate (limited by disk RAM array expansion).	High scalability with optimized query speed.
Query Speed	Slower (requires complex SQL multi-joins).	Lightning fast ($O(1)$ array offset indexing).	Fast for high-level summaries.
Storage Overhead	Low (only stores populated records).	High (sparse matrix indexing required).	Balanced.

Q2. Explain the Bottom-Up Computation (BUC) Algorithm for Iceberg Cubes. (8 Marks)

An **Iceberg Cube** computes only those cuboid cells whose aggregate measure satisfies an iceberg condition (e.g. `HAVING COUNT(*) >= min_support`).
BUC Algorithm: Computes data cubes from bottom to top (from 1-D apex children down to base cuboids). It exploits the *Apriori anti-monotonicity property*: If an aggregate count of a cell in a parent cuboid fails `min_support`, none of its descendant child cells can satisfy the condition! BUC immediately prunes the entire subtree beneath that cell, eliminating massive wasteful computations!

Topic 29.1: Advanced Dimensional Modeling & Indexing Strategies

Indexing Architecture	Internal Bitwise Mechanism	Best OLAP Query Type
Bitmap Indexing	Creates a bit vector for each distinct value of a low-cardinality attribute (e.g. `Gender: {M: 1010, F: 0101}`). Queries execute via hardware-accelerated bitwise AND/OR/NOT operations.	Low-cardinality categorical filters (`WHERE Region = 'East' AND Status = 'Active'`).
Join Indexing	Maintains index relationships between foreign keys of fact tables and primary keys of dimension tables across multi-table joins without physical table joining.	Star schema star-join query acceleration.
Bitmapped Join Index	Stores bitmap representations of dimension attributes directly inside the fact table index.	Complex multi-attribute enterprise OLAP reporting.

🏛️ **Step-by-Step Solved Problem: Bitmap Index Bitwise Query Processing**

A customer table has 6 records. Attribute **Gender** has distinct values $\{M, F\}$; Attribute **CustType** has distinct values $\{VIP, Regular\}$:

- Gender Bitmaps: $\text{Bit}_M = [1, 0, 1, 1, 0, 0], \text{Bit}_F = [0, 1, 0, 0, 1, 1]$.
- CustType Bitmaps: $\text{Bit}_{VIP} = [1, 1, 0, 1, 0, 0], \text{Bit}_{Regular} = [0, 0, 1, 0, 1, 1]$.

Query: Find all Female customers who are VIP (`Gender = 'F' AND CustType = 'VIP'`):

$$\text{Result} = \text{Bit}_{\mathbf{F}} \wedge \text{Bit}_{\mathbf{VIP}} = [\mathbf{0, 1, 0, 0, 1, 1}] \text{ AND } [\mathbf{1, 1, 0, 1, 0, 0}] = [\mathbf{0, 1, 0, 0, 0, 0}]$$

$$\mathbf{\text{Interpretation: Record index 2 (Customer 2) is the exact matching tuple! (Computed in 1 CPU cycle!)}}}$$

Q3. Explain Slowly Changing Dimensions (SCD Type 1, Type 2, and Type 3). (8 Marks)

In a data warehouse, dimension attribute values change over time (e.g. customer relocates from Chicago to Dallas):
• **SCD Type 1 (Overwrite):** Overwrites the old value with the new value. Fast, but *destroys historical context* (past sales in Chicago are falsely credited to Dallas).
• **SCD Type 2 (Add New Row):** Creates a new row with a new surrogate key, marking the old row with `end_date` and `is_current = FALSE`. Preserves complete historical audit trail (industry gold standard).
• **SCD Type 3 (Add New Column):** Adds a `current_city` and `previous_city` column to the existing row. Tracks only the immediate prior state.

Topic 29.2: Advanced Star-Cubing & Materialized View Optimization

🏛️ Step-by-Step Solved Problem: Materialized View Selection using the Greedy (HRU) Algorithm

A data cube lattice has base cuboid V_0 (cost = 100). The costs of computing each dependent cuboid view and their lattice ancestor dependencies are given below. Select top $k = 2$ views to materialize to maximize total query evaluation benefit:

- Views: V_1 (cost = 50, ancestors: V_0), V_2 (cost = 30, ancestors: V_0), V_3 (cost = 20, ancestors: V_1, V_2), V_4 (cost = 10, ancestors: V_3).

Step 1: Calculate Benefit of materializing first view:

- Benefit(V_1): Saves $(100 - 50) = 50$ for V_1; saves $(100 - 50) = 50$ for descendant V_3; saves 50 for $V_4 \implies B(V_1) = 50 + 50 + 50 = \mathbf{150}$.
- Benefit(V_2): Saves $(100 - 30) = 70$ for V_2; saves $(100 - 30) = 70$ for V_3; saves 70 for $V_4 \implies B(V_2) = 70 + 70 + 70 = \mathbf{210}$.
- Benefit(V_3): Saves $(100 - 20) = 80$ for V_3; saves 80 for $V_4 \implies B(V_3) = 80 + 80 = \mathbf{160}$.

$$\text{Choice 1: Select } \mathbf{V_2} \text{ with Maximum Benefit } = \mathbf{210}!$$

Step 2: Calculate Marginal Benefit given V_2 is already materialized (cost of V_3, V_4 now reduced to 30):

- Benefit($V_1 \mid V_2$): Saves $(100 - 50) = 50$ for V_1; saves $(30 - 30) = 0$ for $V_3, V_4 \implies B(V_1 \mid V_2) = \mathbf{50}$.
- Benefit($V_3 \mid V_2$): Saves $(30 - 20) = 10$ for V_3; saves $(30 - 20) = 10$ for $V_4 \implies B(V_3 \mid V_2) = 10 + 10 = \mathbf{20}$.
- Benefit($V_4 \mid V_2$): Saves $(30 - 10) = 20$ for $V_4 \implies B(V_4 \mid V_2) = \mathbf{20}$.

$$\text{Choice 2: Select } \mathbf{V_1} \text{ with Marginal Benefit } = \mathbf{50}!$$

$$\text{Final Optimal Views to Materialize: } \{\mathbf{V_2}, \mathbf{V_1}\} \quad (\text{Total Benefit } = \mathbf{210} + \mathbf{50} = \mathbf{260})$$

Topic 29.3: Master University Exam Problem Bank (Part II)

🏛️ Solved Problem 3: Data Cube Aggregation with the Multi-Way Array Aggregation Algorithm

The **Multi-Way Array Aggregation (Zhao et al. 1997)** algorithm computes a full multidimensional MOLAP data cube in a single pass over memory arrays:

1. Partition the multidimensional array into chunks that fit in RAM cache.
2. Order dimensions such that the dimension with the largest size/cardinality is placed first ($D_1 \times D_2 \times D_3$).
3. Scan chunk by chunk, aggregating along 2D planes and 1D lines simultaneously without re-reading chunks from disk!
4. Minimizes total memory requirement to $O(|D_2 \times D_3| + |D_3|)$ rather than the full $O(|D_1 \times D_2 \times D_3|)$!

Q4. Compare Enterprise Data Warehouse, Data Mart, and Virtual Warehouse. (8 Marks)

• **Enterprise Data Warehouse (EDW):** An enterprise-wide, unified repository collecting data across all functional business departments (Finance, Marketing, HR, Operations). Requires extensive corporate schema design.
• **Data Mart:** A subset of enterprise data focused on a single department or specific business line (e.g., Marketing Data Mart). Can be dependent (sourced directly from EDW) or independent (sourced directly from operational DBs).
• **Virtual Warehouse:** A set of relational views over operational transactional databases. Zero physical storage required, but severely degrades operational OLTP performance when complex queries execute!

Q5. Explain Surrogate Keys in Data Warehousing and why operational primary keys should NEVER be used. (6 Marks)

A **Surrogate Key** is an artificial, system-generated integer primary key (e.g., `1001, 1002`) used in dimension tables.
• **Why Not Operational PKs?** (1) Natural operational keys change over time (e.g. employee SSN re-issued), (2) Heterogeneous source systems use conflicting PK formats, and (3) Tracking Slowly Changing Dimensions (SCD Type 2) requires multiple historical rows for the same operational entity (which violates operational PK uniqueness!).

Topic 29.4: Master University Exam Problem Bank (Part III)

🏛️ Solved Problem 4: Star-Cubing Shared-Prefix Tree Construction

Star-Cubing (Xu et al. 2001) integrates top-down and bottom-up data cube computation with star-tree structures:

- Compresses identical attribute prefixes into single tree paths.
- If a subtree's aggregate count satisfies `min_support`, compute all descendant cuboid aggregations simultaneously.
- If an aggregate fails `min_support`, prune the entire star-tree branch (star-node reduction).
- Achieves up to $\mathbf{100}\times$ speedup over naive array cubing algorithms!

Q6. Compare Fact Tables vs Dimension Tables in Data Warehouse Design. (8 Marks)

Dimension	Fact Table	Dimension Table
Contents	Numeric additive business measurements (revenue, quantities) and composite FKs.	Descriptive qualitative textual context (names, categories, addresses).
Growth & Size	Massive (millions to billions of rows; grows continuously).	Small to moderate (hundreds to thousands of rows).
Granularity	Defined at atomic leaf transaction level.	Hierarchical roll-up levels (Store $\rightarrow$ City $\rightarrow$ State $\rightarrow$ Country).
Structure	Tall, narrow (few columns, massive rows).	Short, wide (many descriptive textual columns).

Topic 29.5: Master University Exam Problem Bank (Part IV)

🏛️ Solved Problem 5: Fact Table Sizing & Storage Capacity Estimation

A national retail chain with 500 physical stores sells 10,000 distinct products. On average, each store processes 2,000 sales transactions per day with an average of 4 items per transaction:

- Daily sales fact records = $500 \text{ stores} \times 2,000 \text{ transactions} \times 4 \text{ items} = \mathbf{4,000,000} \text{ rows/day}$.
- Annual fact table growth = $4,000,000 \times 365 = \mathbf{1,460,000,000} \text{ rows/year}$ $(1.46 \text{ Billion rows/year})$.
- Each row consists of 5 integer FKs ($4 \text{ bytes} \times 5 = 20 \text{ bytes}$) and 3 float measures ($8 \text{ bytes} \times 3 = 24 \text{ bytes}$) + row overhead ($16 \text{ bytes}$) = $\mathbf{60}$ bytes/row.
- Annual raw storage = $1.46 \times 10^9 \times 60 \approx \mathbf{87.6}$ GB/year. With B-tree & Bitmap indexing ($2.5\times$ multiplier) $\implies$ $\mathbf{219}$ GB/year!

Q7. Compare Kimball's Dimensional Bus Architecture with Inmon's Corporate Information Factory (CIF). (10 Marks)

Dimension	Bill Inmon (Top-Down CIF)	Ralph Kimball (Bottom-Up Bus)
Central Architecture	Single enterprise normalized relational DW (3NF). Data marts are departmental downstream extracts.	No centralized 3NF EDW. The warehouse is the *union of all dimensional star-schema data marts*.
Integration Mechanism	Enterprise Data Model (EDM).	**Conformed Dimensions** and Conformed Facts.
Implementation Speed	Slow initial ramp-up (high upfront engineering cost).	Fast iterative delivery; agile business value per mart.
End-User Access	Users query data marts, not the core 3NF EDW.	Users query dimensional star schemas directly.

Q8. Detail the Mechanics of Materialized Query Tables (MQT) and Automatic Query Rewrite. (8 Marks)

A **Materialized Query Table (MQT / Indexed View)** physically stores pre-aggregated query results on disk. When an end-user submits a complex SQL aggregation query (e.g. `SELECT Year, Region, SUM(Sales)...`), the RDBMS **Cost-Based Query Optimizer** automatically rewrites the execution plan to read directly from the compact MQT rather than scanning the massive 1.5-billion-row base fact table, reducing query execution latency from minutes to milliseconds!

Topic 29.6: Complete Solved Laboratory Problem Bank on OLAP Aggregations

🏛️ Solved Numerical 6: Bitmap Index Vector Compression via Run-Length Encoding (WAH)

A high-cardinality bitmap index bit vector has 128 bits containing sparse ones: `00000000 00000000 ... 00000001`:

- **Word-Aligned Hybrid (WAH) Compression:** Replaces consecutive sequences of 31-bit zero words with a single 32-bit run word encoding the count.
- Compresses a 100-million-bit sparse bitmap into less than $200\ \text{KB}$ of RAM!
- Allows bitwise AND/OR queries to execute directly on the compressed format without decompression!

🏛️ Solved Problem 9: Incremental View Maintenance in Real-Time Data Warehouses

When operational databases insert new delta records ΔR:

- **Counting Algorithm (Gupta et al.):** For view $V = \pi_A(\sigma_C(R \bowtie S))$, maintain a count column $count(*)$ in the materialized view.
- When a base tuple is deleted, decrement $count(*)$. Only delete the view row when $count(*) = 0$.
- Allows real-time incremental view refreshment without re-computing the full multi-table join!

🏛️ Solved Problem 10: Star-Cubing Shared-Prefix Path Algorithm

Constructs a tree where nodes represent attribute-value pairs and links represent prefix sharing. Performs recursive top-down cuboid lattice traversal with bottom-up aggregation!

🏛️ Solved Numerical 10: Multi-Tiered Data Warehouse Physical Storage Layouts

In columnar analytical data warehouses (Snowflake, BigQuery, ClickHouse), data is stored column-by-column rather than row-by-row:

- **Columnar Compression:** High compression ratios ($10\times$) due to identical data types in single columns (dictionary encoding, run-length encoding).
- **I/O Vectorization:** Analytical queries (`SELECT SUM(Sales)...`) read strictly the required columns from disk, completely skipping all other unreferenced columns, achieving $100\times$ faster throughput than traditional row stores!

Q9. Compare Star Schema with Constellation Schema for Multi-Departmental Enterprises. (8 Marks)

A single Star Schema contains only one central fact table, suitable for isolated departmental data marts (e.g. Sales). An enterprise-wide system requires a **Fact Constellation (Galaxy) Schema** with multiple fact tables (e.g., `Sales_Fact`, `Inventory_Fact`, `Shipping_Fact`) interconnected via shared **Conformed Dimensions** (`Dim_Date`, `Dim_Product`, `Dim_Store`), preventing fragmented analytical silos and enabling cross-functional enterprise intelligence!

🏛️ Solved Problem 12: Factless Fact Table Applications in Business Intelligence

A **Factless Fact Table** contains NO numeric measurement facts (no dollars or quantities); it contains ONLY foreign keys pointing to dimension tables:

- **Event Tracking:** Tracking university student class attendance: `(student_key, course_key, date_key, room_key)`. The fact of the event occurring is the measure!
- **Coverage Table:** Tracking promotional marketing campaigns: `(product_key, store_key, promo_key, date_key)` to determine which products were on promotion in which stores, even if zero units were sold!

🏛️ Solved Problem 14: Semi-Additive vs Additive vs Non-Additive Measure Design

In a financial banking data warehouse:

- **`Transaction_Amount`:** Additive across Account, Branch, and Time ($\sum \text{Amount}$ is meaningful).
- **`Account_Ending_Balance`: Semi-Additive** across Account and Branch, but NON-additive across Time (must use `Current_Balance` snapshot or `Avg_Balance` over time).
- **`Loan_Interest_Rate_%`: Non-Additive** across all dimensions (must compute weighted average: $\frac{\sum \text{Balance} \times \text{Rate}}{\sum \text{Balance}}$).

🏛️ Solved Problem 15: Slowly Changing Dimensions Type 4 (History Table) vs Type 6 (Hybrid)

• **SCD Type 4:** Maintains a primary dimension table with current attributes, and writes historical changes to a separate history fact table.
• **SCD Type 6 (1 + 2 + 3):** Hybrid schema containing both historical rows (Type 2) and current overwrite columns (Type 3) alongside surrogate keys!

🏛️ Solved Problem 16: Star-Tree Construction Algorithm & Iceberg Condition Pruning

In Star-Cubing, each node N stores a star-table and aggregate counts. If $\text{count}(N) < \text{min\_support}$, the star-tree node is dropped and all child cuboid branches are pruned in a single operation!